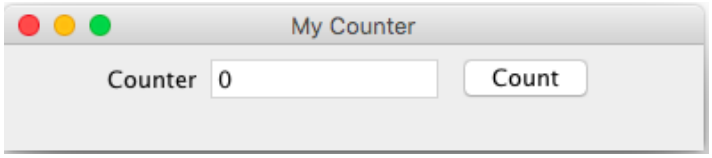
Tutorial – Week 10 5COSC019W – Object Oriented Programming – Java

Event Handler, JUnit (from the previous week) and Exception Handling

24-25 November

GUI application

1) Write a Swing GUI application called Counter, as shown in the figure below. Every time the Count button is clicked, the counter value should increase by 1.



The user interface has 3 JComponents:

- JLabel
- JButton
- JTextField

The Swing Components are to be added onto the ContentPane of the top-level container JFrame. You can retrieve the ContentPane via the method getContentPane() from a JFrame.

Note: Swing Components are kept in package javax.swing. They begin with a prefix "J", e.g., JButton, JLabel, JFrame.

```
import java.awt.*;       // Using AWT's layouts
import javax.swing.*;    // Using Swing components and containers
```

Write a class Counter that extends JFrame:

```
public class Counter extends JFrame{

   private JLabel lblCount;      // Declare component JLabel
   private JTextField tfCount;   // Declare component JTextField
   private JButton btnCount;     // Declare component JButton
   private int count = 0;       // counter's value

    // Constructor to setup UI components and event handlers
   public Counter () {
       // sets layout to FlowLayout, which arranges
       // Components from left-to-right, then top-to-bottom.
      Container cp = getContentPane();
      cp.setLayout(new FlowLayout());

      lblCount = new JLabel("Counter"); // Construct component Label
      add(lblCount);                    // "super" Frame adds Label

      tfCount = new JTextField(count + "", 10); // Construct component TextField
      tfCount.setEditable(false);               // read-only
      add(tfCount);                             // "super" Frame adds TextField

      btnCount = new JButton("Count");   // Construct component Button
      add(btnCount);                     // "super" Frame adds Button
   }
```

Instance variables

Implement Constructor

Get content Pane and set the layout

Add compo-nent to the content pane

Event Handling

The ***first step*** is to write an inner class that `implements EventListener`.

- An inner class called MyListener is defined to be used as the listener for the ActionEvent fired by the Button btnCount. Since MyListener is an ActionEvent listener, it has to implement the ActionListener interface and provide an implementation for the `actionPerformed()` method declared in the interface.

- Even if instance variables `tfCount`,and `count` are private, the inner class `MyListener` has access to them. This is the reason why an inner class is used instead of an ordinary outer class.

In the same file, implement the listener:

```
private class MyListener implements ActionListener {
      @Override
      public void actionPerformed(ActionEvent evt) {
         ++count;
         tfCount.setText(count + "");
      }
   }
```

Every time an ActionEvent is generated the count variable is incremented by 1 and the value is displayed in the GUI

The ***second step*** is to register the listener to the JComponent: in your code, you need to add the following lines in the Counter constructor

```
MyListener handler = new MyListener();
btnCount.addActionListener(handler);
```

- btnCount is the **source** object that fires ActionEvent when clicked
- The source add "handler" instance as an ActionEvent listener, which provides an ActionEvent handler called actionPerformed().
- Clicking btnCount invokes actionPerformed().

2) Modify the previous code in order to include two additional buttons for counting down and resetting the count value.

Hints:

- Add the other two JButtons to the content pane and register the Event Listener
- You can use `evt.getSource()` to retrieve which component has fired the event

```
public void actionPerformed(ActionEvent evt) {

            // checking which is the source that fired the event
            if (evt.getSource() == btnCount) {
                ++count;
                tfCount.setText(count + "");

            } else if (evt.getSource() == btnCountDown) {
                --count;
                tfCount.setText(count + "");
            } else if (evt.getSource() == btnReset) {
                count = 0;
                tfCount.setText(count + "");
            }
        }
```

JUnit Exercise

6) For this exercise, you can use the **Date class** that was provided in tutorial week 03. You can download the NetBeans project from BB or from the following link:

https://learning.westminster.ac.uk/ultra/courses/_102630_1/outline/edit/document/_5621406_1?courseId=_102630_1&view=content&state=view

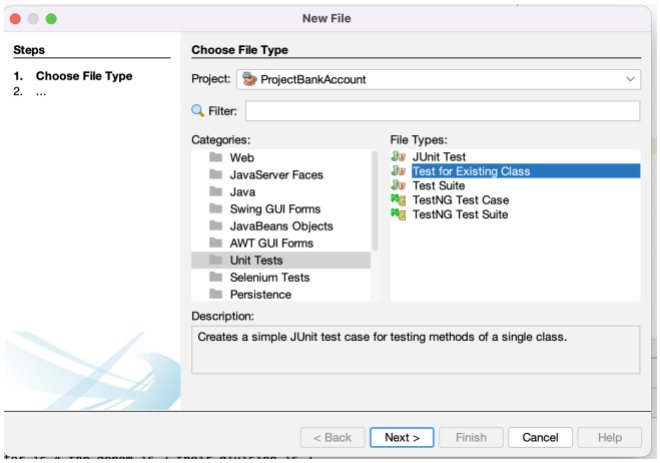
Your task is to write JUnit tests to test the following methods in the class Date:

- Constructor
- Set and get year methods
- Set and get month methods
- Set and get day methods

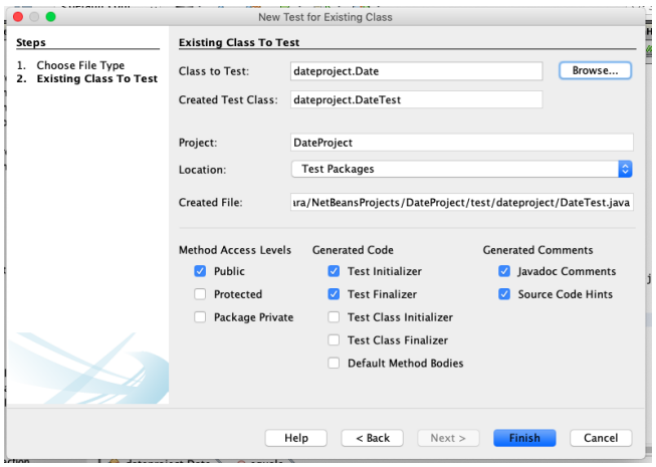
Some guidance on creating unit tests:

Writing Java and Test Classes Using NetBeans IDE:

1)Select the Java project "DateProject". Right click on the selected Java project. Select **New -> Other.** Select **Unit Tests** and then **Test for Existing Class**



1) Choose the class to test (in our case Date as we want to test the methods from this class) and tick or untick the check boxes as shown in the figure below. Press Finish.



The test class will be generated. You can keep only the first part, which corresponds to the constructor, the setUp and tearDown. You can delete all the other methods as we will build our tests.

```
public class DateTest {

    public DateTest() {
    }

    @BeforeEach
    public void setUp() {
    }

    @AfterEach
    public void tearDown() {
    }

}
```

@BeforeEach is an annotation that tells JUnit to run the setUp method before each test.

@AfterEach is an annotation to define a tearDown method, which runs after each test is completed. If it's empty, means no specific cleanup is needed.

Since we want to test methods on a Date object, let's create an instance of the class Date that will be used in each test method:

```
public class DateTest {

    private Date date;

    public DateTest() {
    }

    @BeforeEach
    public void setUp() {
    }

    @AfterEach
    public void tearDown() {
    }

}
```

We will instantiate the object Date inside the setUp() method. This ensure that a new Date instance is created before every test, with some initial values:

```
public class DateTest {

    private Date date;

    public DateTest() {
    }

    @BeforeEach
    public void setUp() {

       date = new Date(18, 11, 2024);
    }

    @AfterEach
    public void tearDown() {
    }

}
```

2) Testing the constructor

```
// test the constructor
    @Test // indicates that is a test
    public void testConstructor() {
        assertEquals(18, date.getDay());
        assertEquals(11, date.getMonth());
        assertEquals(2024, date.getYear());
    }
```

This method checks if the Date object's day, month and year attributes are correctly set by the constructor.

assertEquals compares the expected value (e.g., 18) with the actual value returned by the getter method doctor.getDay(). If they are equal, the test passes; if not, it fails.

3) Testing the Getter and Setter for Day

```
@Test
    public void testSetAndGetDay() {
        date.setDay(20);
        assertEquals(20, date.getDay());
    }
```

This test checks the setDay and getDay methods.
`date.setDay(20)` sets the day to be 20, and assertEquals confirms that this value is correctly stored and returned by `date.getDay());`

4) Implement by yourself the tests for the Getter and Setter methods for month and year

Exception Handling

5) Try to compile the following code:

```
public static void main(String[] args) {

      int myArray[] = new int[5];
      // trying to access element 5
      System.out.println(myArray[5]);
   }
```

Why do you get the error?
When you try to compile and run the above program, the message "IndexArrayOutOfBoundsException" is printed to the standard output. This happens because you try to access index 5, which does not exist. Remember, the index in an array can be from 0 to N-1, where N is the length of the array.

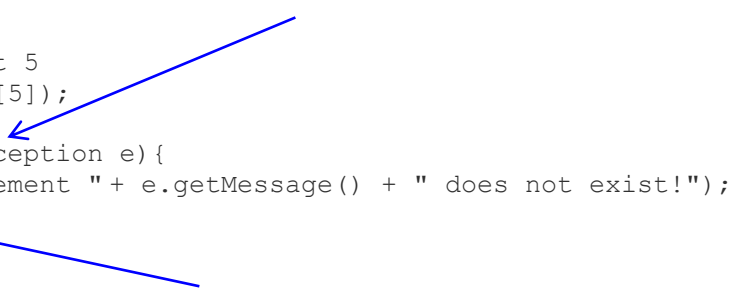
2) Using try and catch blocks, add some code to change the previous uncontrolled standard message to the following customised message:

`Message: "The element 5 does not exist!".`

Try on your own before checking the following solution.

```
public class MyClass{

   public static void main(String args[]){
      try{
         int myArray[] = new int[5];
         // trying to access element 5
         System.out.println(myArray[5]);
      }
      catch(ArrayIndexOutOfBoundsException e){
         System.out.println("The element " + e.getMessage() + " does not exist!");
      }
   }
```



```
}
```

You handle the error

4) Consider the following code:

```
public static void main(String[] args) {


        Scanner input = new Scanner(System.in);

        int value = 0;
        System.out.println("Please enter an integer");
        value = input.nextInt();

        System.out.println("Value: " + value);
    }
```

If, instead of an integer, the user inserts a String, an error will be displayed. Modify the code in order to validate the user input and handle the error in case characters are inserted instead of numbers.

5) **Custome Exception**

Implement a custom exception that will be thrown if a processed number is negative.

Try first to implement the following steps on your own. Check then the code solution.

1. Create a Custom Exception:

- Create a Java Project called CustomExceptionProject.
- Create a Java class named NegativeNumberException that extends Exception.
- Implement a parameterised constructor that takes a message as an argument and calls the superclass constructor with that message.

2. Use the Custom Exception:

- Create another class named NumberProcessor.
- Implement an instance method named processNumber that takes an integer as an argument.
- In the processNumber method, check if the provided number is negative.
- If the number is negative, throw an instance of NegativeNumberException with an appropriate error message (e.g. “Negative number not allowed”).
- The method processNumber must throw the exception in its declaration.

3. Test the Custom Exception:

- In the main method of your project, create an instance of NumberProcessor.
- Call the processNumber method with both positive and negative numbers.

- Handle the exception appropriately using a try-catch block.